

Chapter 10: Exception Handling

Object Oriented Programming Language (JAVA)

Md. Eamin Rahman



What Is an Exception?

- An **exception** is an ***abnormal condition*** that occurs during **program execution**.
- It represents a **run-time error** that disrupts the normal flow of a program.
- Languages without exception handling rely on **manual error checking**, such as ***error codes***.
- Java handles **run-time errors** in a clean, **object-oriented** way.

Exception-Handling Fundamentals (Overview)

- A Java **exception** is an **object** that represents an *abnormal condition* occurring at **run time**.
- When an error occurs, an exception object is **created and thrown**.
- The exception may be **handled locally** or **passed to another method**.
- Eventually, the exception is **caught and processed**.

Sources of Exceptions

- Exceptions may be generated by the **Java run-time system**.
- **System-generated** exceptions indicate **language rule violations** or **runtime environment errors**.
- Exceptions can also be **manually generated** by user code.
- **Manually thrown** exceptions are typically used to **report errors to a method's caller**.

Exception-Handling Keywords

- Java **exception** handling uses **five keywords**:
  - `try`
  - `catch`
  - `throw`
  - `throws`
  - `finally`
- These **keywords** work together to **detect, propagate, and handle** exceptions.

try–catch–finally Blocks

- Code that may cause an **exception** is placed inside a `try` block.
- If an **exception** occurs, it is **thrown** and can be handled using `catch`.
- **System-generated exceptions** are automatically thrown by the **Java run-time system**.
- To **manually throw** an exception, use the `throw` keyword.

try–catch–finally Blocks

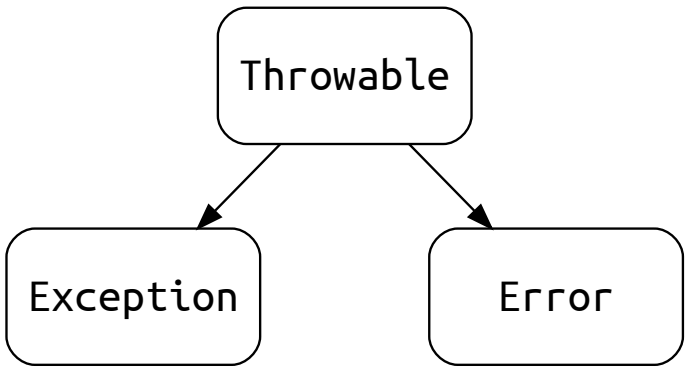
- Any exception that is ***passed out*** of a method must be declared using a `throws` clause.
- Multiple `catch` blocks may be used for different **exception** types.
- Code that **must always execute** is placed in a `finally` block.

General Exception-Handling Form

```
try {
    // code to monitor for exceptions
}
catch (ExceptionType1 exOb) {
    // handler for ExceptionType1
}
catch (ExceptionType2 exOb) {
    // handler for ExceptionType2
}
finally {
    // code that always executes
}
```

Exception Types (Hierarchy)

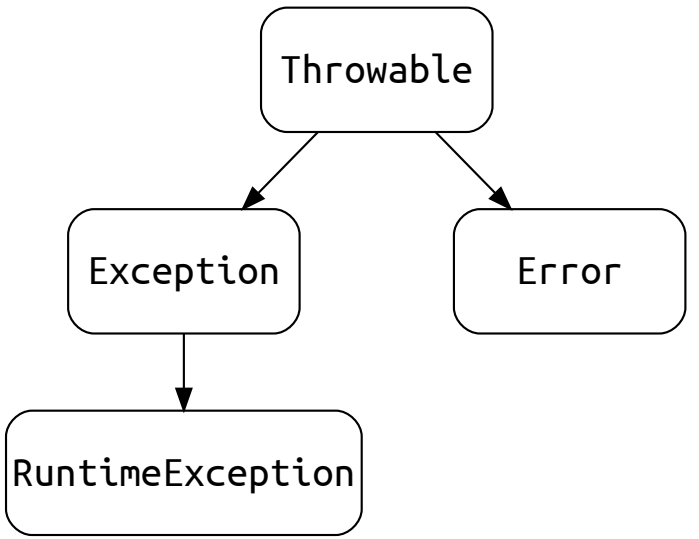
- **All** exception types are **subclasses** of the built-in class `Throwable`.
- `Throwable` sits at the top of the exception **class hierarchy**.



- Directly below `Throwable` are **two main branches** that categorize exceptions.
- One **branch** is headed by `Exception`.
- The other **branch** is headed by `Error`.

Exception Class

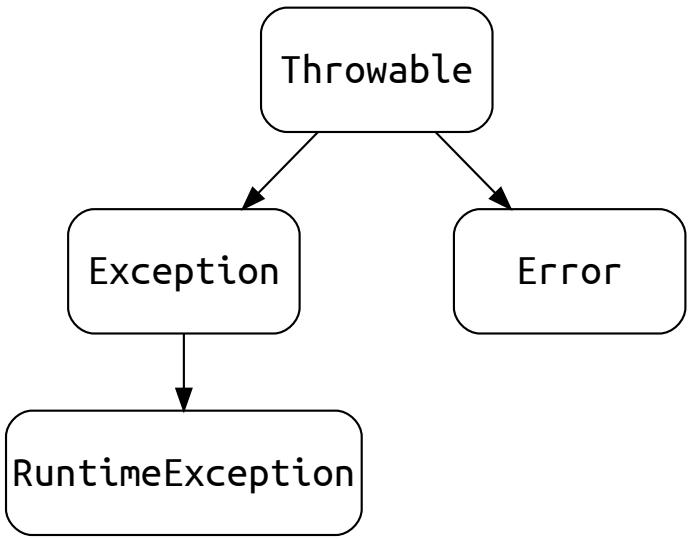
- `Exception` class represents conditions that **user programs are expected to catch**.
- **Custom** exceptions are created by **subclassing** `Exception`.



- A key subclass of `Exception` is `RuntimeException`, which are **automatically** defined.
- Includes **common** programming **errors** such as **division by zero** and **invalid array indexing**.

Error Class

- `Error` class represents serious problems related to the **Java run-time environment**.
- These errors are **not expected to be caught**.



- **Stack overflow** is a typical example.
- **Errors** usually result from **catastrophic failures** and are not the focus of this chapter.

Uncaught Exceptions (Concept)

- An **uncaught exception** occurs when a program does not provide a handler.
- **Java run-time system** detects the error and **automatically creates and throws** an exception object.
- When an **exception** is thrown, **normal execution stops immediately**.
- The **exception** must be caught and handled, or the **program terminates**.
- If no handler is provided, the **exception** is processed by Java's **default exception handler**.

Uncaught Exception Example

```
1 class Exc0 {
2     public static void main(String[] args) {
3         int d = 0;
4         int a = 42 / d;
5     }
6 }
```

- A ***divide-by-zero*** causes the **Java run-time system** to create and throw an **exception**.
- **No user-defined handler** is present in this program.

Default Exception Handler

- **Uncaught exceptions** are handled by the **default run-time handler**.
- The default handler:
  - Displays the **exception type and message**
  - Prints a **stack trace**
  - **Terminates the program**

Console Output

```
java.lang.ArithmeticException: / by zero
at Exc0.main(Exc0.java:4)
```

Stack Trace and Debugging

- The **stack trace** shows the **sequence of method calls** leading to the **error**.
- It includes:
  - **Class** name
  - **Method** name
  - **File** name
  - **Line** number

Stack Trace Example

```java
class Exc1 {
    static void subroutine() {
        int d = 0;
        int a = 10 / d;
    }
    public static void main(String[] args) {
        Exc1.subroutine();
    }
}
```

Console Output

```
java.lang.ArithmeticException: / by zero
at Exc1.subroutine(Exc1.java:4)
at Exc1.main(Exc1.java:7)
```

- **Stack traces** are essential for **debugging**, as they pinpoint where and how the **error** occurred.

Benefits of Own Exception Handler

- By providing your own **exception handler**, you can:
  - **Handle exceptions** in a way that is appropriate for your program.
  - **Prevent** the program from **automatically terminating**.
  - **Allow** the program to **continue running** after an **exception** occurs.
  - Provide **more informative error messages** to the user.

Using try and catch (Basic Structure)

- To guard against a ***run-time error,*** enclose the risky code inside a `try` block.
- Immediately follow the `try` block with one or more `catch` clauses.
- A `try` and its `catch` blocks form a **single unit**.
- The **exception** type to be handled is specified in the `catch` clause.

Using try and catch (Rules & Scope)

- The scope of a `catch` block is limited to the **immediately preceding** `try` block.
- A `catch` block **cannot catch exceptions** thrown by another `try` block.
- (An exception exists for **nested try blocks**, discussed later.)
- **Statements** protected by `try` must be enclosed in **curly braces**.
- A `try` block **cannot be used with a single statement**.

Basic try–catch Example

```java
class Exc2 {
    public static void main(String[] args) {
        int d, a;
        try {
            d = 0;
            a = 42 / d;
            IO.println("This will not be printed.");
        }
        catch (ArithmeticException e) {
            IO.println("Division by zero is not allowed.");
        }
        IO.println("After catch block.");
    }
}
```

Console Output

```
Division by zero is not allowed.
After catch block.
```

Another try–catch Example

```java
import java.util.Random;
class HandleError {
    public static void main(String[] args) {
        int a=0, b=0, c=0;
        Random r = new Random();
        for(int i=0; i<32000; i++) {
            try {
                b = r.nextInt();
                c = r.nextInt();
                a = 12345 / (b/c);
            } catch (ArithmeticException e) {
                IO.println("Division by zero.");
                a = 0; // set a to zero and continue
            }
            IO.println("a: " + a);
        }
    }
}
```

Displaying an Exception Description

- The class `Throwable` overrides the `toString()` method.
- `toString()` returns a **string description** of the exception.
- You can print this description by passing the exception object to `println()`.

Example: Printing an Exception

- The `catch` block in the preceding program can be rewritten like this:

```
1  catch (ArithmeticException e) {
2      IO.println("Exception: " + e);
3      a = 0; // set a to zero and continue
4  }
```

- When executed, a divide-by-zero error produces:

Console Output

```
Exception: java.lang.ArithmeticException: / by zero
```

Multiple `catch` Clauses (Concept)

- A **single block** of code may generate **more than one type of exception**.
- To handle this, you can include **multiple `catch` clauses**.
- Each `catch` block handles a **different exception type**.

Multiple `catch` Clauses (Execution Flow)

- When an exception is thrown, `catch` blocks are checked **in order**.
- The **first matching** `catch` block is executed.
- Once a `catch` block executes, the remaining `catch` blocks are **skipped**.
- Execution then continues **after the try–catch structure**.

Multiple `catch` Clauses (Example)

```
1  class MultipleCatches {
2      public static void main(String[] args) {
3          try {
4              int b = 42 / args.length;
5              int[] c = { 1 };
6              c[42] = 99;
7          } catch(ArithmeticException e) {
8              IO.println("Divide by 0: " + e);
9          } catch(ArrayIndexOutOfBoundsException e) {
10             IO.println("Array index oob: " + e);
11         }
12         IO.println("After try/catch blocks.");
13     }
14 }
```

Multiple `catch` Clauses (Example)

```
1  class MultipleCatches {
2      public static void main(String[] args) {
3          try {
4              int b = 42 / args.length;
5              int[] c = { 1 };
6              c[42] = 99;
7          } catch(ArithmeticException e) {
8              IO.println("Divide by 0: " + e);
9          } catch(ArrayIndexOutOfBoundsException e) {
10             IO.println("Array index oob: " + e);
11         }
12         IO.println("After try/catch blocks.");
13     }
14 }
```

Console Output: `java MultipleCatches`

```
Divide by 0: java.lang.ArithmeticException: / by zero
After try/catch blocks.
```

Multiple `catch` Clauses (Example)

```java
class MultipleCatches {
    public static void main(String[] args) {
        try {
            int b = 42 / args.length;
            int[] c = { 1 };
            c[42] = 99;
        } catch(ArithmeticException e) {
            IO.println("Divide by 0: " + e);
        } catch(ArrayIndexOutOfBoundsException e) {
            IO.println("Array index oob: " + e);
        }
        IO.println("After try/catch blocks.");
    }
}
```

Console Output: `java MultipleCatches One`

```
Array index oob: java.lang.ArrayIndexOutOfBoundsException: Index 42 out of bounds for length 1
After try/catch blocks.
```

Multiple `catch` Clauses (Order Matters)

- When using multiple `catch` blocks, **subclasses must appear before their superclasses**.
- A `catch` block for a **superclass** can catch **its subclasses as well**.
- Therefore, placing a **superclass** ***first*** would prevent a **subclass** `catch` from ever executing.
- This would lead to **unreachable code**, which is a **compile-time error** in Java.

Multiple `catch` (Unreachable Code)

```java
class SuperSubCatch {
    public static void main(String[] args) {
        try {
        int a = 0;
        int b = 42 / a;
        } catch(Exception e) {
            IO.println("Generic Exception catch.");
        }
        catch(ArithmeticException e) { // ERROR - unreachable
            IO.println("This is never reached.");
        }
    }
}
```

Console Output

```
SuperSubCatch.java:9: error: exception ArithmeticException has already been caught
        catch(ArithmeticException e) { // ERROR - unreachable
        ^
1 error
```

Nested `try` Statements (Concept)

- The `try` statement can be **nested**.
- A `try` statement can be placed **inside another** `try` **block**.
- Each time a `try` block is entered, its **exception context is pushed** onto the **stack**.

Nested `try` Statements (Exception Propagation)

- If an **inner `try` block** does not handle an exception, the **stack is unwound**.
- The **outer `try`** block's `catch` clauses are then checked.
- This process ***continues*** until:
  - A matching `catch` block is found, or
  - All nested `try` blocks are **exhausted**.
- If **no match** is found, the **default run-time handler** processes the **exception**.



Nested `try` Statements (Example)

```java
class NestTry {
    public static void main(String[] args) {
        try {
            int a = args.length; int b = 42 / a;
            IO.println("a = " + a);
            try {
                if(a==1) a = a/(a-a); // division by zero
                if(a==2) { int[] c = { 1 }; c[42] = 99;  }
            } catch(ArrayIndexOutOfBoundsException e) {
                IO.println("Array index out-of-bounds: " + e);
            }
        } catch(ArithmeticException e) {
            IO.println("Divide by 0: " + e);
        }
    }
}
```

Nested `try` Statements (Example)

```java
class NestTry {
    public static void main(String[] args) {
        try {
            int a = args.length; int b = 42 / a;
            IO.println("a = " + a);
            try {
                if(a==1) a = a/(a-a); // division by zero
                if(a==2) { int[] c = { 1 }; c[42] = 99;  }
            } catch(ArrayIndexOutOfBoundsException e) {
                IO.println("Array index out-of-bounds: " + e);
            }
        } catch(ArithmeticException e) {
            IO.println("Divide by 0: " + e);
        }
    }
}
```

Console Output: `java NestTry`

```
Divide by 0: java.lang.ArithmeticException: / by zero
```

Nested `try` Statements (Example)

```java
class NestTry {
    public static void main(String[] args) {
        try {
            int a = args.length; int b = 42 / a;
            IO.println("a = " + a);
            try {
                if(a==1) a = a/(a-a); // division by zero
                if(a==2) { int[] c = { 1 }; c[42] = 99;  }
            } catch(ArrayIndexOutOfBoundsException e) {
                IO.println("Array index out-of-bounds: " + e);
            }
        } catch(ArithmeticException e) {
            IO.println("Divide by 0: " + e);
        }
    }
}
```

Console Output: `java NestTry One`

```
a = 1
Divide by 0: java.lang.ArithmeticException: / by zero
```

Nested `try` Statements (Example)

```java
class NestTry {
    public static void main(String[] args) {
        try {
            int a = args.length; int b = 42 / a;
            IO.println("a = " + a);
            try {
                if(a==1) a = a/(a-a); // division by zero
                if(a==2) { int[] c = { 1 }; c[42] = 99;  }
            } catch(ArrayIndexOutOfBoundsException e) {
                IO.println("Array index out-of-bounds: " + e);
            }
        } catch(ArithmeticException e) {
            IO.println("Divide by 0: " + e);
        }
    }
}
```

Console Output: `java NestTry One Two`

```
a = 2
Array index out-of-bounds: java.lang.ArrayIndexOutOfBoundsException: Index 42 out of bounds for length 1
```

Nested `try` Statements (Example)

```java
class MethNestTry {
    static void nesttry(int a) {
        try {
            if(a==1) a = a/(a-a); // division by zero
            if(a==2) { int[] c = { 1 }; c[42] = 99; }
        } catch(ArrayIndexOutOfBoundsException e) {
            IO.println("Array index out-of-bounds: " + e);
        }
    }
    public static void main(String[] args) {
        try {
            int a = args.length; int b = 42 / a;
            IO.println("a = " + a);
            nesttry(a);
        } catch(ArithmeticException e) {
            IO.println("Divide by 0: " + e);
        }
    }
}
```

The `throw` Statement (Purpose)

- You can **explicitly throw** an exception using the `throw` statement.
- General form:

```
throw ThrowableInstance;
```

- `ThrowableInstance` must be an **object** of type `Throwable` or its **subclass**.
- **Primitive** types (e.g., `int`, `char`) and **non-Throwable** classes (e.g., `String`) **cannot be thrown**.

Obtaining a Throwable Object

- A `Throwable` **object** can be obtained in **two ways**:
  - From a `catch` **parameter**
  - By creating a new **object** using the `new` operator
- Example:

```
throw new ArithmeticException("Divide by zero");
```

Execution Flow After `throw`

- Execution **stops immediately** after a `throw` statement.
- The **nearest** enclosing `try` block is checked for a matching `catch` block.
- If **no match** is found, the search continues **outward**.
- If **no matching handler** exists, the **default run-time handler**:
  - Prints the **stack trace**
  - **Terminates** the program

Example: Using `throw`

```
1  class ThrowDemo {
2      static void demoproc() {
3          try {
4              throw new NullPointerException("demo");
5          } catch(NullPointerException e) {
6              IO.println("Caught inside demoproc.");
7              throw e; // rethrow the exception
8          }
9      }
10     public static void main(String[] args) {
11         try {
12             demoproc();
13         } catch(NullPointerException e) {
14             IO.println("Recaught: " + e);
15         }
16     }
17 }
```

The `throws` Clause (Purpose)

- A `throws` clause declares the **exceptions** that a method may **throw**.
- It is required for all **exceptions** *except*:
  - `Error`
  - `RuntimeException`
  - Their subclasses
- If a required **exception** is not declared, a **compile-time error** occurs.

Syntax of `throws`

```
returnType methodName(parameterList) throws exception-list {
    // method body
}
```

- The `exception-list` is a **comma-separated list** of exception types.
- It informs the **caller** that the **method may** propagate these **exceptions**.

Not Using `throws`

```java
class ThrowsDemo {
    static void throwOne() {
        IO.println("Inside throwOne.");
        throw new IllegalAccessException("demo");
    }
    public static void main(String[] args) {
        throwOne();
    }
}
```

Console Output

```
ThrowsDemo.java:4: error: unreported exception IllegalAccessException; must be caught or declared to be thrown
        throw new IllegalAccessException("demo");
              ^
1 error
```

Using `throws` + Caught in `main`

```java
class ThrowsDemo {
    static void throwOne() throws IllegalAccessException {
        IO.println("Inside throwOne.");
        throw new IllegalAccessException("demo");
    }
    public static void main(String[] args) {
        try {
            throwOne();
        } catch(IllegalAccessException e) {
            IO.println("Caught: " + e);
        }
    }
}
```

Console Output

```
Inside throwOne.
Caught: java.lang.IllegalAccessException: demo
```

Using `throws` + Caught by JVM

```java
class ThrowsDemo {
    static void throwOne() throws IllegalAccessException {
        IO.println("Inside throwOne.");
        throw new IllegalAccessException("demo");
    }
    public static void main(String[] args) throws IllegalAccessException {
        throwOne();
    }
}
```

Console Output

```
Inside throwOne.
Exception in thread "main" java.lang.IllegalAccessException: demo
        at ThrowsDemo.throwOne(ThrowsDemo.java:4)
        at ThrowsDemo.main(ThrowsDemo.java:7)
```

Declared but Not Thrown

```java
class ThrowsDemo {
    static void throwOne() throws IllegalAccessException {
        IO.println("Inside throwOne.");
    }
    public static void main(String[] args) throws IllegalAccessException {
        throwOne();
    }
}
```

Console Output

```
Inside throwOne.
```

Unchecked Exception in `throws`

```java
class ThrowsDemo {
    static void throwOne() throws ArithmeticException {
        IO.println("Inside throwOne.");
    }
    public static void main(String[] args) throws IllegalAccessException {
        throwOne();
    }
}
```

Console Output

```
Inside throwOne.
```

Unchecked Exception Thrown

```java
class ThrowsDemo {
    static void throwOne() throws ArithmeticException {
        IO.println("Inside throwOne.");
        throw new ArithmeticException("demo");
    }
    public static void main(String[] args) throws IllegalAccessException {
        throwOne();
    }
}
```

Console Output

```
Inside throwOne.
Exception in thread "main" java.lang.ArithmeticException: demo
        at ThrowsDemo.throwOne(ThrowsDemo.java:4)
        at ThrowsDemo.main(ThrowsDemo.java:7)
```

The `finally` Block (Purpose)

- `finally` defines a block that executes after `try` / `catch`.
- It runs **before** the code that follows the `try` / `catch`.
- The `finally` clause is **optional**.
- Each `try` must have at least:
  - One `catch`, or
  - One `finally`

When Does `finally` Execute?

- **Executes** whether or not an **exception** is thrown.
- **Executes** even if:
  - No `catch` matches the **exception**.
  - A `return` statement is executed inside `try` or `catch`.
- **Runs** just **before** the method **returns** to the caller.



Why Use `finally`?

- Commonly used for **resource cleanup**, such as:
  - **Closing file** handles
  - **Releasing network** connections
  - **Freeing allocated** resources
- **Ensures cleanup** code runs **regardless of exceptions**

Example: Using `finally`

```
1  class FinallyDemo {
2      static void procA() {
3          try { IO.println("inside procA");
4              throw new RuntimeException("demo");
5          } finally { IO.println("procA's finally"); }
6      }
7      static void procB() {
8          try { IO.println("inside procB"); return;
9          } finally { IO.println("procB's finally"); }
10     }
11     static void procC() {
12         try { IO.println("inside procC");
13         } finally { IO.println("procC's finally"); }
14     }
15     public static void main(String[] args) {
16         try { procA(); }
17         catch (Exception e) { IO.println("Exception caught"); }
18         procB();          procC();
19     }
20 }
```

Example: Using `finally`

```
1  class FinallyDemo {
2      static void procA() {
3          try { IO.println("inside procA");
4              throw new RuntimeException("demo");
5          } finally { IO.println("procA's finally"); }
6      }
7      static void procB() {
8          try { IO.println("inside procB"); return;
9          } finally { IO.println("procB's finally"); }
10     }
11     static void procC() {
12         try { IO.println("inside procC");
13         } finally { IO.println("procC's finally"); }
14     }
15     public static void main(String[] args) {
16         try { procA(); }
17         catch (Exception e) { IO.println("Exception caught"); }
18         procB();          procC();
19     }
20 }
```

Console Output

```
inside procA
procA's finally
Exception caught
inside procB
procB's finally
inside procC
procC's finally
```

Java's Built-in Exceptions

- **Java** defines many **exception** classes in the `java.lang` package.
- These **built-in exceptions** are available **automatically** (*no import required*).
- Two main categories:
  - **Unchecked exceptions** (subclasses of `RuntimeException`)
  - **Checked exceptions** (not subclasses of `RuntimeException`)

Unchecked Exceptions

- Subclasses of `RuntimeException`
- Do **NOT** need to be declared in a `throws` clause.
- The **compiler does not check** whether they are handled.
- Called **unchecked exceptions**

Checked Exceptions

- **Exceptions** that are **not subclasses** of `RuntimeException`
- Must be:
  - Handled using `try-catch`, or
  - Declared using `throws`
- If **not** handled or declared → **compile-time error**
- Called **checked exceptions**

Unchecked Exceptions (1/4)

Exception	Meaning
`ArithmeticException`	Arithmetic error, such as divide-by-zero.
`ArrayIndexOutOfBoundsException`	Array index is out-of-bounds.
`ArrayStoreException`	Assignment to an array element of an incompatible type.
`ClassCastException`	Invalid cast.
`EnumConstantNotPresentException`	An attempt is made to use an undefined enumeration value.



Unchecked Exceptions (2/4)

Exception	Meaning
`IllegalArgumentException`	Illegal argument used to invoke a method.
`IllegalCallerException`	A method cannot be legally executed by the calling code.
`IllegalMonitorStateException`	Illegal monitor operation, such as waiting on an unlocked thread.
`IllegalStateException`	Environment or application is in incorrect state.
`IllegalThreadStateException`	Requested operation not compatible with current thread state.



Unchecked Exceptions (3/4)

Exception	Meaning
`IndexOutOfBoundsException`	Some type of index is out-of-bounds.
`LayerInstantiationException`	A module layer cannot be created.
`NegativeArraySizeException`	Array created with a negative size.
`NullPointerException`	Invalid use of a null reference.
`NumberFormatException`	Invalid conversion of a string to a numeric format.



Unchecked Exceptions (4/4)

Exception	Meaning
`SecurityException`	Attempt to violate security.
`StringIndexOutOfBoundsException`	Attempt to index outside the bounds of a string.
`TypeNotPresentException`	Type not found.
`UnsupportedOperationException`	An unsupported operation was encountered.

Checked Exceptions (1/2)

Exception	Meaning
`ClassNotFoundException`	Class not found.
`CloneNotSupportedException`	Attempt to clone an object that does not implement the Cloneable interface.
`IllegalAccessException`	Access to a class is denied.
`InstantiationException`	Attempt to create an object of an abstract class or interface.

Checked Exceptions (2/2)

Exception	Meaning
`InterruptedException`	One thread has been interrupted by another thread.
`NoSuchFieldException`	A requested field does not exist.
`NoSuchMethodException`	A requested method does not exist.
`ReflectiveOperationException`	Superclass of reflection-related exceptions.

Creating Your Own Exception Subclasses (Purpose)

- Java's **built-in exceptions** handle most **common errors**.
- You can create your **own exception** types for **application-specific** situations.
- Define a **subclass** of `Exception` (which is a **subclass** of `Throwable`).
- Your **subclass** does **not** need to **implement** anything.
- Its existence in the **type system** allows it to be used as an **exception**.

Inheritance of Methods

- The `Exception` class does **not define any methods** of its own.
- It **inherits** methods from `Throwable`.
- All **exceptions**, including user-defined ones, have the **methods** defined by `Throwable`.
- You may **override** one or more of these **methods** in your custom **exception** classes.

Constructors of `Exception`

- `Exception` defines four public constructors.
- **Two** support chained **exceptions**.
- The **other two** are:
  - `Exception()`
  - `Exception(String msg)`
- The **first** form creates an **exception** with no description.
- The **second** form lets you specify a **description**.

Overriding `toString()`

- Providing a **description** is often useful.
- Sometimes it is better to **override** `toString()`.
- The `Throwable` version of `toString()`:
  - Displays the **exception name**
  - Followed by a **colon**
  - Then the **description**
- By overriding `toString()`, you can prevent the **exception name** and **colon** from being displayed.

Example: Custom Exception

```
1  class MyException extends Exception {}
2  class ExceptionDemo {
3      static void compute(int a) throws MyException {
4          IO.println("Called compute(" + a + ")");
5          if(a > 10) throw new MyException();
6          IO.println("Normal exit");
7      }
8      public static void main(String[] args) {
9          try {
10             compute(1);
11             compute(20);
12         } catch (MyException e) {
13             IO.println("Caught " + e);
14         }
15     }
16 }
```

Example: Custom Exception

```java
class MyException extends Exception {}
class ExceptionDemo {
    static void compute(int a) throws MyException {
        IO.println("Called compute(" + a + ")");
        if(a > 10) throw new MyException();
        IO.println("Normal exit");
    }
    public static void main(String[] args) {
        try {
            compute(1);
            compute(20);
        } catch (MyException e) {
            IO.println("Caught " + e);
        }
    }
}
```

Console Output

```
Called compute(1)
Normal exit
Called compute(20)
Caught MyException
```

Example: Custom Exception with Overridden `toString()`

```
1  class MyException extends Exception {
2      private int detail;
3      MyException(int a) { detail = a; }
4      public String toString() {
5          return "Exception for value " + "[" + detail + "]";
6      }
7  }
```

Example: Custom Exception with Overridden `toString()`

```
1  class ExceptionDemo {
2      static void compute(int a) throws MyException {
3          IO.println("Called compute(" + a + ")");
4          if(a > 10) throw new MyException(a);
5          IO.println("Normal exit");
6      }
7      public static void main(String[] args) {
8          try {
9              compute(1);
10             compute(20);
11         } catch (MyException e) {
12             IO.println("Caught " + e);
13         }
14     }
15 }
```

Example: Custom Exception with Overridden `toString()`

```
1  class ExceptionDemo {
2      static void compute(int a) throws MyException {
3          IO.println("Called compute(" + a + ")");
4          if(a > 10) throw new MyException(a);
5          IO.println("Normal exit");
6      }
7      public static void main(String[] args) {
8          try {
9              compute(1);
10             compute(20);
11         } catch (MyException e) {
12             IO.println("Caught " + e);
13         }
14     }
15 }
```

Console Output

```
Called compute(1)
Normal exit
Called compute(20)
Caught MyException for value [20]
```

Chained Exceptions (Concept)

- **Chained exceptions** allow you to associate one **exception** with another.
- The **second exception** describes the **cause** of the **first exception**.
- Useful when layers of **exceptions** exist.

Why Use Chained Exceptions?

- Example:
  - A **method throws** an `ArithmeticException` (divide by zero).
  - The real cause was an **I/O error**.
- The method must **throw** `ArithmeticException`.
- But you may also want to **inform** the **caller** that the underlying cause was an **I/O error**.
- **Chained exceptions** allow this.

Constructors Supporting Chaining

- **Two constructors** were added to `Throwable`:
  - `Throwable(Throwable causeExc)`
  - `Throwable(String msg, Throwable causeExc)`
- `causeExc` is the **underlying reason** for the **exception**.
- The **second form** allows **both**:
  - A description (`msg`)
  - A cause exception

Chained Exception Methods

- **Methods** added to `Throwable` for *chained exceptions*:
  - `Throwable getCause()`
  - `Throwable initCause(Throwable causeExc)`
- `getCause()` returns the **underlying exception**.
  - Returns `null` if none exists.
- `initCause()` associates a **cause** with an **exception**.
- Returns a **reference** to the **exception**.

Important Rules About `initCause()`

- The **cause** can be ***set only once***.
- You can call `initCause()` **only once** per **exception object**.
- If the **cause** was set using a **constructor**,
  - You **cannot set it again** using `initCause()`.
- `initCause()` is mainly used for **legacy exception** classes that do not support the **chaining constructors**.

Example: Chained Exceptions

```java
class ChainedExcDemo {
    static void demoproc() {
        NullPointerException e = new NullPointerException("top layer");
        e.initCause(new ArithmeticException("cause"));
        throw e;
    }
    public static void main(String[] args) {
        try {
            demoproc();
        } catch(NullPointerException e) {
            IO.println("Caught: " + e);
            IO.println("Original cause: " + e.getCause());
        }
    }
}
```

Console Output

```
Caught: java.lang.NullPointerException: top layer
Original cause: java.lang.ArithmeticException: cause
```

Three Additional Exception Features (JDK 7)

Beginning with **JDK 7**, ***three*** useful **features** were added:

- **Try-with-resources**
- **Multi-catch**
- **More precise rethrow**

1 Try-with-Resources

- **Automates** the process of **releasing resources**.
- Commonly used for **files** or other **closeable resources**.
- Based on an **expanded form** of the `try` statement.
- Ensures **resources** are **closed** when no longer needed.
- Described in detail when **files** are introduced.

2 Multi-Catch

- Allows **two or more exceptions** to be caught by a **single** `catch` clause.
- **Prevents code duplication** when handlers use the **same code**. Syntax:

```
catch (NullPointerException | ArithmeticException ex) {
    // handler code
}
```

- **Exception** types are **separated** by the **OR operator** `|`.
- The **multi-catch** parameter is **implicitly** `final`.
- It **cannot be reassigned** inside the catch block.

3 More Precise Rethrow

- Also called *final rethrow*.
- **Restricts rethrown exceptions** to only those:
  - **Thrown** by the associated `try` block
  - Not handled by a preceding `catch`
  - That are a **subtype** or **supertype** of the parameter
  - Mainly affects **checked exceptions**
- For this **feature** to apply:
  - The **catch** parameter must be **effectively final**, or
  - **Explicitly** declared `final`

What Is Rethrow?

- **Rethrow** means throwing an **exception** again from inside a `catch` block.
- The **exception** is **caught**.
- Some action is performed (e.g., logging).
- The same **exception** is **thrown again**.
- It propagates to the caller.
- The **compiler** assumes `e` is of type `Exception`.
- The **method** must declare: `throws Exception`

Example: Rethrow (Before JDK 7 Behavior)

```
 1  class OpenException extends Exception { }
 2  class CloseException extends Exception { }
 3  class RethrowExcDemo {
 4      public static void main(String args[]) throws Exception {
 5          boolean flag = args.length/2==0;
 6          try {
 7              if (flag) throw new OpenException();
 8              else throw new CloseException();
 9          }
10          catch (Exception e) {
11              IO.println(e);
12              throw e;
13          }
14      }
15  }
```

What Is More Precise Rethrow? (JDK 7)

- The **compiler** analyzes the `try` block.
- It determines the actual **checked exceptions thrown**.
- **Allows** to throw **specific exceptions** along with their **supertype**.
- **Rethrow** type = **actual checked exceptions** thrown in try.
- Where as **before JDK 7**, **rethrow** type = **catch parameter type**.

Example: More Precise Rethrow (JDK 7+)

```
1  class OpenException extends Exception { }
2  class CloseException extends Exception { }
3  public class RethrowExcDemo {
4      public static void main(String args[]) throws OpenException, CloseException {
5          boolean flag = args.length/2==0;
6          try {
7              if (flag) throw new OpenException();
8              else throw new CloseException();
9          }
10         catch (Exception e) {
11             IO.println(e);
12             throw e;
13         }
14     }
15 }
```